

SE3002 Assignment 02 — Structural Testing of PX4-Autopilot v1.17.0

1. Group details and baseline

Student	Name	Roll number	Primary area
S1	Taimoor Shaukat	24i3015	environment, baseline, coverage, gaps
S2	Muhammad Bilal Tahir	24i3166	DataValidator + DataValidatorGroup, MC/DC
S3	Ali	24i3158	FailureDetector/Injector, findings, report

Section B, SE3002 Software Quality Engineering, Assignment 02.

Baseline: tag `v1.17.0`, commit `d6f12ad1c4f70ad3230afd7d86e971421e02fef4` (`git rev-parse 'v1.17.0^{commit}'`; the annotated tag object `a5eb12d2...` is a different git object), all submodules initialised. All student work is in test files and their CMake registration; **no production file was modified** (`work/production_change_log.md`; `deliverables/patch_files.txt` lists the 16 patched files: 12 `Sqe*Test.cpp`, 4 `CMakeLists.txt`).

2. Local environment and setup evidence

Three local environments were used, all Ubuntu 24.04 / x86_64 / GCC 13.3.0 / lcov 2.0-1:

Environment	Toolchain set-up	Used for
E1 native Ubuntu 24.04.4, i7-1355U, 30 GiB	user-local cmake/ninja (pip), lcov built from source — no sudo was available (<code>work/DECISIONS.md D-002</code>)	original development, iterations IT-1..IT-4
E2 Windows 11 WSL2, Ubuntu 24.04.5	official <code>Tools/setup/ubuntu.sh --no-nuttx --no-sim-tools</code>	independent reproduction of the original suite (identical hit sets)
E3 Windows 11 WSL2, Ubuntu 24.04.5, Ryzen 7 7840HS, 7 GiB; cmake 3.28.3, ninja 1.11.1	official <code>Tools/setup/ubuntu.sh --no-nuttx --no-sim-tools</code>	revision v2: fresh clone, extended scope, all final evidence (<code>evidence/v2/</code>)

Build: `PX4_CMAKE_BUILD_TYPE=Coverage make tests TESTFILTER=_no_tests` (config `px4_sitl_test`, `-O0`, `--coverage`). Upstream baseline on E3: 147 upstream tests pass; the only other entry, `antlr4_tests_NOT_BUILT`, is a CTest placeholder for an optional dependency that `ubuntu.sh` does not install (`evidence/v2/coverage/baseline/ctest.log`). Upstream `make tests_coverage` was not used because it runs `make clean` and drops branch data; the plain `ctest` + `lcov` commands are in Appendix A.

3. Repository analysis and scope selection record

The course clarification asked for *substantial* business/control logic rather than a convenient module. The scope was therefore extended in this revision from four to seven production files, forming three safety chains. Candidates were measured against the live source (`work/scope/CANDIDATES.md`); rejected candidates and reasons are listed there (e.g. `commander/failsafe` already has an upstream test and is too large for the time box; `Safety.cpp` too small).

Area	File (responsibility)	Size: source / executable lines, decisions (compound)	Key dependencies	Test level
A	<code>sensors/data_validator/DataValidator.cpp</code> — confidence of one sensor stream (timeout, stale, error density)	155 / 58, 15 (0)	none (explicit arguments)	GTest unit
B	<code>sensors/data_validator/DataValidatorGroup.cpp</code> — redundant-sensor voting, failover classification (MC/DC component)	347 / 155, 38 (5)	owns DataValidators	GTest unit

Area	File (responsibility)	Size: source / executable lines, decisions (compound)	Key dependencies	Test level
C	commander/failure_detector/FailureDetector.cpp — attitude, ESC, motor, imbalance, external-ATS failure flags for Commander's failsafe	354 / 153, 44 (14)	8 params, 6 uORB topics	GTest functional
D	commander/failure_detector/FailureInjector.cpp — injected motor/ESC failures	134 / 62, 13 (4)	SYS_FAILURE_EN, vehicle_command	GTest functional
E	lib/battery/battery.cpp — state of charge, LOW/CRITICAL/EMERGENCY warning, RLS internal-resistance estimator, remaining time	434 / 231, 35 (13)	10 params, vehicle_status, flight_phase_estimation	GTest functional
F	land_detector/LandDetector.cpp + MulticopterLandDetector.cpp — ground contact → maybe landed → landed, free fall, ground effect, at-rest, flight time (drives auto-disarm)	583 / 238, 59 (23)	14 params, 13 uORB topics, work queue, wall-clock hysteresis	GTest functional (+ real work queue)

Total: **2 007 source lines, 897 executable lines, 204 decisions, 59 compound decisions, 863 source-level branch outcomes**. Chains: sensor redundancy (A+B), failure detection (C+D), power and landing safety (E+F). Per-file decision inventories with line numbers and covering tests: `work/basis/INVENTORY_*.md` (Battery and LandDetector added in v2). Upstream coverage of the scope before any student test: Areas C, D, F 0 %; A 74.1 % lines; B 65.2 %; E 60.6 % (battery code runs incidentally when the upstream `sitl-*` tests boot the px4 binary; no upstream GTest targets it).

Test-level justification. A and B have no parameters or uORB, so unit level is sufficient. C, D and E read parameters at construction and use uORB, which `gtest_functional_main` initialises. For F the decision methods are protected and `LandDetector::Run()` is private and inherited through a *private* base (`px4::ScheduledWorkItem`), so no test can call it; `Run()` is therefore executed through PX4's real work-queue manager (the module's own `start()` path) and observed on uORB. SITL was not needed: no decision in the scope depends on simulator dynamics.

Exclusions. Header-only one-line accessors; the orphaned `data_validator/tests/` directory (never built); other land detectors (fixed-wing, VTOL, rover, airship) — the multicopter detector is the one used by multicopters and shares the base class with them. `print()` methods stay in statement/branch scope.

4. Testing approach and test doubles

Decisions were derived from the source into the inventories before tests were written; every test header names the decision outcomes it targets and has Given/When/Then lines, from which the workbook's Test Inventory is generated (`work/inventory/test_inventory.csv`, checked 1:1 against the code by `tools/sqe_trace_check.py`: "TRACEABILITY OK, 212 tests"). Oracles are return values, getters, published uORB messages and parameters, derived from comments, parameter descriptions or independent arithmetic (e.g. hand-computed filter steps, a synthetic battery model) — never the production expression copied into the test.

Test doubles used (the course clarification explicitly asks for them):

Technique	Where	Purpose
Test subclass (seam) re-exporting protected members, no override	<code>TestableBattery</code> , <code>TestableMcLandDetector</code> , <code>RunnableMcLandDetector</code>	call protected <code>updateParams()</code> / decision methods, set the input; <code>Run()</code> would fill, stop a item
Link-time stub <code>-Wl,--wrap=param_get</code>	<code>functional-SqeFailureInjectorParamFault</code>	make the <code>SYS_FAILURE</code> read fail (closes former gap G-06)
Link-time fake of <code>uORB::Manager::orb_data_copy</code> (<code>--wrap</code> on the mangled symbol), one-shot per topic	<code>functional-SqeFailureDetectorCopyFault</code> , <code>functional-SqeBattery</code>	"copy() fails right after updated()" (closes former gap G-07; battery L356)

Technique	Where	Purpose
Replacement global operator new returning nullptr once	unit-SqeDataValidatorGroupAlloc	NuttX-style allocation f (closed G-01, confirmed F-13)
Real collaborators instead of mocks	uORB, parameter store, PX4 work queue	functional realism where determinism allows it

Determinism. Hysteresis in the land detector is driven with explicit timestamps; the battery takes timestamps as arguments. Where wall-clock time is unavoidable (FailureDetector hysteresis, work-queue tests, the production 8 s minimum-thrust hysteresis) waits are lower bounds only and outputs are polled until a predicate holds, so machine load can delay but not falsify a result. Every "stays false" test has a positive control. Boundary values are tested at the exact threshold wherever the comparison is reachable (e.g. battery warning thresholds, 2 s settling time, land detector distance/velocity limits, DataValidator 1e-6 equality).

Independence and repeatability. 12 binaries, 210 active tests (+2 disabled sanitizer probes): A 21, B 38, C 45, D 16, E 31, F 59. All 12 binaries pass 10 shuffled repeats (evidence/v2/tests/shuffle/), every test passes run alone (evidence/v2/tests/individual.log: pass=210 fail=0), and `ctest -R Sqe` passes 12/12 (evidence/v2/tests/ctest_sqe.log). A final dry run from the submission zip itself — `fresh git clone --branch v1.17.0 --recursive, git apply` of the zipped patch, Coverage build, `ctest -R Sqe, lcov` — passed 12/12 and produced a coverage summary identical to the submitted one (evidence/v2/dryrun/dryrun.log: "IDENTICAL COVERAGE SUMMARY").

5. MC/DC component selection and interpretation

Primary component: DataValidatorGroup. `get_best()` decides which physical IMU/magnetometer feeds the estimator, and `failover_index()/failover_state()` report why a failover happened (call sites `voted_sensors_update.cpp`, `VehicleMagnetometer.cpp`). The critical behaviour analysed is *selection and failover classification of the active redundant sensor*. Following the clarification that a two-condition decision is not automatically substantive, only decisions with ≥ 3 conditions that directly implement this behaviour were selected: DVG-D13 candidate switch (7 conditions, L187-190), DVG-D14 best-changed (3, L203), DVG-D15 failsafe vs. priority preference (3, L207-208), DVG-D32/D34 failover reporting (3 each, L282-283/L301-302). Simpler guards in the file (null checks, loop bounds, two-operand tests) were deliberately excluded from MC/DC and covered at branch level.

29 matrix rows over 22 tests give a unique-cause independence pair for each of the 19 conditions, in the short-circuit form (a condition not evaluated in one test of a pair is recorded `NE(x)` and is not counted as "changed"); every condition is evaluated True and False and every decision has both outcomes. Infeasible combinations are proved, not omitted ($B \Rightarrow G$, $F \Rightarrow D$, $(A \wedge B) \Rightarrow C$ in D13; $J \Rightarrow F \Rightarrow I \Rightarrow T$ in D14). The K pair of D15 has no observable effect on any public output ($K = \text{False}$ only on the first call, where `true_failsafe` is never read, and `reset_state()` is then a no-op), so it is evidenced structurally: line 210 runs 0 times for SQE-DVG-MC-21 and once for SQE-DVG-MC-01 (evidence/coverage/pertest/MC21_03_EVIDENCE.md).

Supplementary MC/DC (land detector). The single most safety-relevant compound decisions in the extended scope were also analysed because a wrong "landed" verdict in flight can trigger auto-disarm: MLD-D35 *maybe landed* (7 conditions, L287-290, $E = \text{vertical_estimate}$ appears coupled as E and $\neg E$), MLD-D29 *ground contact* (5, L249-251) and MLD-D37 *ground effect* (5, L301-303; S and U read the same takeoff state). 21 tests (SQE-MLD-MC-01...21) give unique-cause pairs for all 17 conditions; the coupled condition E is shown independent by MC-09/MC-13 (F and G each evaluated in only one test of the pair, with equal logical values). The 8 s minimum-thrust term (G) is exercised against the production hysteresis (MC-14).

`tools/sqe_mcdc_check.py work/mcdc/mcdc_matrix.csv` → **ALL DECISIONS COMPLETE (8 decisions, 50 rows)**; the matrix was also re-checked by an independent script that re-evaluates each row's Boolean formula (0 errors). Mutation analysis (§7) adds that all 16 compile-valid `&&/||` mutants of the DataValidatorGroup MC/DC decisions are detected.

6. Coverage analysis (baseline vs final, raw vs source-level)

Captured on E3 with lcov 2.0 from three runs of the same build: upstream only (`ctest -E Sqe`), student only (`ctest -R Sqe`) and all tests. Reports (HTML + `scope.info` + `summary.txt`): `evidence/v2/coverage/{baseline,student,final}/`.

File	Baseline lines	Baseline src-branches	Final lines	Final raw branches	Exception edges	Final source-level branches
DataValidator.cpp	43/58	21/30	58/58	30/30	0	30/30 (100 %)
DataValidatorGroup.cpp	101/155	54/116	155/155	114/116	0	114/116 (98.3 %)
FailureDetector.cpp	0/153	0/190	153/153	190/254	64	190/190 (100 %)
FailureInjector.cpp	0/62	0/48	62/62	48/57	9	48/48 (100 %)
battery.cpp	140/231	68/179	231/231	178/248	69	178/179 (99.4 %)
LandDetector.cpp	0/122	0/144	122/122	143/210	66	143/144 (99.3 %)
MulticopterLandDetector.cpp	0/116	0/156	116/116	156/192	36	156/156 (100 %)
Scope	284/897 (31.7 %)	143/863 (16.6 %)	897/897 (100 %)	859/1107 (77.6 %)	244 (0 hit)	859/863 (99.5 %)

The student-only capture gives exactly the final numbers, so the student suite alone produces all of this coverage. "Raw" is lcov's figure including the exception-unwind edges GCC inserts after calls when exceptions are enabled (block IDs `e...`); "source-level" counts only the decision/operand outcomes written in the source (`tools/sqe_branch_split.py`). gcov branch coverage is operand-level for `&&/||` (one pair per evaluated operand) and is therefore at least condition coverage on every compound decision; MC/DC (§5) is the stronger, separate claim.

Iterations. IT-1...IT-3 (original suite) closed FD-D25/D38/D41; IT-4 closed G-01 (allocation fault double) and G-04 (misdiagnosed `print()` ternary) — `work/coverage_iterations.md`. **IT-5 (this revision)** closed G-06 and G-07 with link-time test doubles, added Areas E and F, and iterated until only proven gaps remained: the first land-detector capture left 5 branches open, of which 4 were closed by new tests (SQE-LDB-01, SQE-LD-11, additions to SQE-MLD-07/17) and 1 was proved infeasible (G-10).

Remaining gaps (proofs and evidence in `work/GAPS.md`; no `LCOV_EXCL` marker, no production change):

- **G-02** `DataValidatorGroup.cpp:78` — compiler-synthesized null test of `delete (_first)` inside `while (_first)`; disassembly in `evidence/v2/gaps/G-02_delete_null_check.txt`.
- **G-03** `DataValidatorGroup.cpp:213` `best != nullptr` False — infeasible: `pre_check_prio` leaves `-1` only in the block that assigns `best`; `best` is a local, unreachable by any test double.
- **G-09** `battery.cpp:130` operand 4 False — infeasible invariant `_internal_resistance_initialized => n_cells > 0` (set only under `n_cells > 0`, reset in the same call whenever `n_cells` changes).
- **G-10** `LandDetector.cpp:187` operand 3 False — infeasible: take-off time can only be set while armed and is reset on the first disarmed cycle.
- **G-05** 244 exception edges — compiler-synthesized; NuttX flight builds use `-fno-exceptions` (`platforms/nuttX/cmake/px4_impl_os.cmake:86`).

7. Test effectiveness — mutation analysis

Coverage proves execution, not detection, so 170 single-operator mutants (153 compile-valid) were run against the suite (`evidence/v2/mutation/MUTATION_SUMMARY.md`). Original scope: 58/68 valid random mutants killed (85.3 %) and 16/16 logical-operator mutants on the MC/DC decisions. New scope: 48/58 (82.8 %). Seven boundary tests were added because of surviving mutants (SQE-DV-21, SQE-FI-14, SQE-BAT-31, SQE-MLD-10/14/19 additions, SQE-MLD-25, SQE-LD-12). All 20 remaining survivors are classified: equivalent (e.g. `index < 1 → <= 1` maps 1 to 1), float values the formulas cannot produce (e.g. confidence exactly 0.9f, F-06), microsecond-exact time equalities, and one out-of-bounds read observable only under a sanitizer.

8. Findings

Confirmed defects (reproducible; full reproduction in `work/findings/REPORT_BLOCKS.md`, `work/FINDINGS.md`):

- **F-07a** `FailureInjector::manipulateEscStatus` loops to `status.esc_count` without clamping to `CONNECTED_ESC_MAX` → ASan stack-buffer-overflow READ at `FailureInjector.cpp:118` (SQE-PRB-05).
- **F-07b** unguarded shift `1 << i_esc` for a non-motor `actuator_function` → UBSan "shift exponent 4294967195" at `FailureInjector.cpp:120` (SQE-PRB-06). Both re-run in this revision (`evidence/v2/sanitizer/`); the 14 active `FailureInjector` tests are clean under both sanitizers. Reachable only with failure injection enabled.
- **F-03** `DataValidatorGroup(0) + add_new_validator()` dereferences a null `_last` (SQE-DVG-10, death test).
- **F-13** constructor crashes when a middle allocation fails on a nullptr-returning allocator (SQE-DVG-AF-03); `add_new_validator()` is robust (SQE-DVG-AF-01). F-03/F-13 are unreachable from production construction sites (`{1}`).

Specification observations and characterizations (behaviour pinned by tests, no stated requirement violated): F-02 non-null stale pointer when `*index == -1` (callers use only the index); F-04 stale counter shared across axes; F-05 `print()` can set the timeout flag; F-06 1 % tie-break float sensitivity; F-08 0-based motor index in one log message; F-09 all-NaN stream keeps confidence 1 (SQE-DV-07); F-10 density exactly at the window gives confidence 0 without a flag (SQE-DV-15); F-11 equal-priority confidence switch counts as failover; F-12 timestamp 0 is a "no data" sentinel; F-14 two `get_best()` effects unobservable; F-15 timed-out motor mask survives disarm; F-16/F-18 redundant guards (G-09, G-10); F-17 flight time includes landed-but-armed time between flights (SQE-LD-03); F-19 non-finite `BAT_AVRG_CURRENT` accepted by the parameter store, handled gracefully (SQE-BAT-25).

Change in this revision: F-02, F-09 and F-10 were previously backed by disabled probes that asserted an *invented* stricter oracle and were shown as FAIL. Because a test oracle must come from a specification and the assignment forbids manufacturing failures, those three probes were removed; the behaviour is reported as observations above. The workbook now shows 210 PASS and 2 FAIL — the two FAILs are the sanitizer-confirmed defects F-07a/F-07b.

9. Gaps, limitations, residual risk, improvements

Limits of the evidence. Unit/functional level only: the scoped code's interaction with its real callers (`VotedSensorsUpdate`, `Commander`, battery drivers) was read, not executed. Wall-clock timing was used for `FailureDetector` and the work-queue tests; one compiler (GCC 13.3) and one optimisation level (`-O0`); no NuttX target, so the allocator and `-fno-exceptions` arguments are by analysis and simulation. Test doubles replace two library functions (`param_get`, `orb_data_copy`) only in dedicated binaries; the rest of the suite runs against the real implementations. The land-detector scope is the multicopter variant; fixed-wing/VTOL/rover detectors are untested.

Residual risk. F-07a/b in failure-injection sessions; behaviour at float boundaries the formulas cannot reach (F-06) under `-O2/-O3`; timing of the 50 ms work-queue cycle under real load.

Improvements. (1) Lockstep/simulated time for `FailureDetector` and the land detector, replacing wall-clock waits; (2) a `-O2` build to measure F-06; (3) sanitizer build types fixed for GCC 13 to run the whole suite under ASan/UBSan; (4) fixes for F-07a/b (`clamp_esc_count`, `guard_i_esc`) — production changes outside this assignment; (5) extend to the fixed-wing and VTOL land detectors, which share the tested base class.

10. Final quality judgment

The evidence supports specific, bounded claims about seven production files — `DataValidator`, `DataValidatorGroup`, `FailureDetector`, `FailureInjector`, battery, `LandDetector` and `MulticopterLandDetector` — and about nothing else in PX4. For these files the 210 student tests execute all 897 executable lines and 859 of the 863 source-level branch outcomes (99.5 %); lcov's raw figure is 77.6 % only because it also counts 244 exception-unwind edges that no call in the scope can take and that flight builds do not contain. Each of the four remaining outcomes has a written proof: one compiler-inserted null check and three logically impossible operand combinations. Two gaps that were earlier called infeasible were closed with link-time test doubles once that technique was applied.

Coverage is backed by stronger evidence than execution. MC/DC holds for the eight analysed compound decisions — DataValidatorGroup's five sensor-selection decisions and the three most safety-relevant land-detector decisions — with a unique-cause independence pair for every one of the 36 conditions. Mutation analysis shows that the assertions detect most injected faults (85 % of valid mutants in the original files, 83 % in the new ones, every logical-operator mutant of the primary MC/DC decisions), and every survivor is classified. All binaries pass ten shuffled repeats and every test passes alone, so results do not depend on order.

This gives high confidence that every reachable decision outcome in the scope was exercised with a checked oracle, and good confidence that wrong outcomes would be noticed. It does not show that the requirements are complete or the code correct for all inputs. The suite confirmed two real defects in FailureInjector (an out-of-bounds read and an undefined shift, found by sanitizers) and two crash paths in DataValidatorGroup that production code cannot reach; several further behaviours are reported as specification observations rather than defects.

The conclusion is limited by test level and environment: the files were tested in isolation from their real callers, with wall-clock rather than simulated time, one compiler at one optimisation level, and without a NuttX target. No claim is made about PX4 as a whole, about other land-detector variants, or about any file outside the analysed scope.

11. AI-assistance record

AI (Claude Code) was used materially in both the original work and this revision: repository navigation, build and coverage troubleshooting, test scaffolding and implementation, MC/DC derivation and cross-checking, gap analysis, mutation-analysis scripts, workbook generation and report drafting. Full log: `work/ai_assistance_log.md`.

Verification practice: every number in this report was re-read from an evidence file produced by a run; every test traces to decision IDs (`tools/sqe_trace_check.py`); the MC/DC matrix was checked by `tools/sqe_mcdc_check.py` and by a separate script; production code was never changed, and mutation analysis was used to catch weak tests (it exposed missing exact-boundary cases and one flawed test that checked at-rest only after the 1 s window had already expired; all fixed, see §7).

Assumptions introduced by AI that the team must own and be able to defend: (1) the extension to battery and land detector and the choice of supplementary MC/DC decisions; (2) the classification of the four remaining gaps as infeasible/compiler-synthesized; (3) reclassifying F-02/F-09/F-10 as observations and removing their probes; (4) that link-time `--wrap` stubs are an acceptable test-double technique for library functions. The revision was produced at a team member's request; that member confirmed assumptions (2)–(4) explicitly (`work/DECISIONS.md` D-020), but it has **not yet been reviewed line-by-line by all three members**; the viva drill log (`work/viva/drill_log.md`) is intentionally empty until the team runs the drills. The team takes responsibility for all targets, expected results, tests and conclusions.

Appendix A — Reproduction commands (E3, official toolchain)

```
git clone --branch v1.17.0 --recursive https://github.com/PX4/PX4-Autopilot.git && cd PX4-Autopilot
git rev-parse 'v1.17.0^{commit}' # d6f12ad1c4f70ad3230afd7d86e971421e02fef4
bash Tools/setup/ubuntu.sh --no-nuttX --no-sim-tools
PX4_CMAKE_BUILD_TYPE=Coverage make tests TESTFILTER=__no_tests__ # build (no tests yet)
cd build/px4_sitl_test
LC="--rc branch_coverage=1 --ignore-errors
mismatch,inconsistent,unused,negative,empty,source,gcov,deprecated"
SCOPE="*/data_validator/DataValidator.cpp */data_validator/DataValidatorGroup.cpp */failure_detector/
FailureDetector.cpp
*/failure_detector/FailureInjector.cpp */lib/battery/battery.cpp */land_detector/LandDetector.cpp
*/land_detector/MulticopterLandDetector.cpp"
lcov $LC --zerocounters -d . && ctest -E Sqe # baseline (upstream only)
lcov $LC --capture -d . -o all.info && lcov $LC --extract all.info $SCOPE -o baseline.info
cd ../.. && git apply 24i3015_24i3166_24i3158_B.patch
PX4_CMAKE_BUILD_TYPE=Coverage make tests TESTFILTER=__no_tests__ # builds the 12 Sqe binaries
cd build/px4_sitl_test
lcov $LC --zerocounters -d . && ctest -R Sqe # student suite (12/12)
lcov $LC --capture -d . -o all.info && lcov $LC --extract all.info $SCOPE -o student.info
genhtml $LC --branch-coverage student.info -o html
./functional-SqeBattery --gtest_shuffle --gtest_repeat=10 # same for every Sqe binary
```

```
# one-command alternative on any Ubuntu 24.04/WSL2 machine: bash tools/sqe_reproduce.sh <submission.zip>
PX4_BUILD=$PWD bash <repo>/tools/sqe_asan_probes.sh; PX4_BUILD=$PWD bash <repo>/tools/sqe_ubsan_probes.sh
python3 <repo>/tools/sqe_branch_split.py student.info # raw vs source-level branches
python3 <repo>/tools/sqe_mcdc_check.py <repo>/work/mcdc/mcdc_matrix.csv
python3 <repo>/tools/sqe_trace_check.py --px4 . --inventory <repo>/work/inventory/test_inventory.csv --mcdc
<repo>/work/mcdc/mcdc_matrix.csv
python3 <repo>/tools/sqe_workbook.py --inventory ... --mcdc ... --xml <repo>/evidence/v2/tests/xml
```

Appendix B — Evidence index

Path	Content
evidence/v2/env/environment.md	E3 tool versions, commit, set-up commands
evidence/v2/coverage/{baseline,student,final}/	scope.info, summary.txt, HTML report, ctest log + JUnit
evidence/v2/tests/xml/	gtest XML per binary (source of the workbook's Execution Result)
evidence/v2/tests/run/, shuffle/, individual.log, ctest_sqe.log	plain runs, 10× shuffled repeats, each test alone, ctest
evidence/v2/sanitizer/	ASan/UBSan runs of SQE-PRB-05/06 and of the active FailureInjector suite
evidence/v2/mutation/	mutation summary and per-mutant records
evidence/v2/gaps/G-02_delete_null_check.txt	disassembly for G-02
evidence/v2/final_run.log	complete log of the final evidence run
evidence/v2/dryrun/	reproduction from the submission zip on a fresh clone (identical coverage summary)
evidence/coverage/..., evidence/repro_independent/	original-scope iterations IT-1...IT-4 and E2 reproduction (history)
work/basis/INVENTORY_*.md, work/basis/SETUP_MAP.md, work/basis/CFG_*.md	structural test basis
work/mcdc/MCDC_ANALYSIS.md, work/mcdc/mcdc_matrix.csv	MC/DC derivation and matrix
work/GAPS.md, work/FINDINGS.md, work/findings/REPORT_BLOCKS.md	gap proofs, findings register
work/inventory/test_inventory.csv	full test-to-decision mapping (212 rows)
deliverables/24i3015_24i3166_24i3158_B.{xlsx,patch}, deliverables/test_source/	workbook, patch, test sources at PX4 paths